الأكاديمية العربية للعلوم والتكنولوجيا والنقل البحري

Arab Academy for Science, Technology & Maritime Transport

Internal Architecture of 8086/8088 Microprocessor

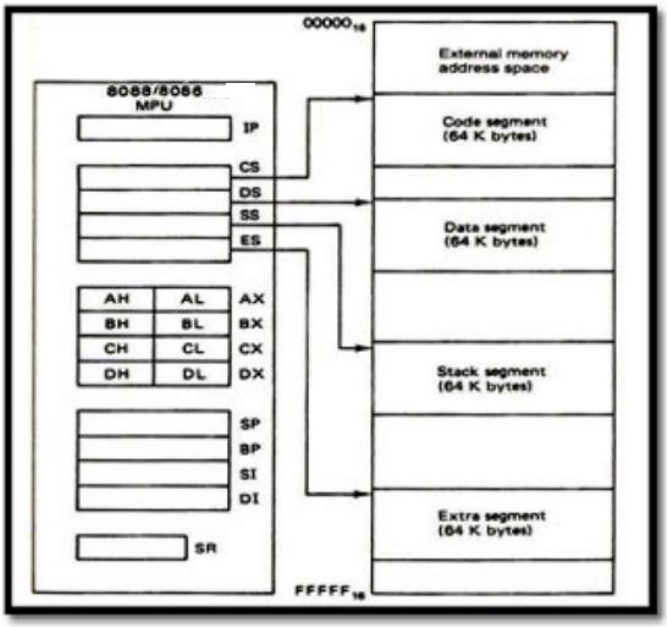
Hand-out of the lecture:

Simple C Program

```
1  #include <stdio.h>
2  main(void)
3  int x,y ;
   :
4  {
5     printf( "Hello message ! \n" );
6
7     return 0;
8  }
```

Data

Code



The **8086/8088 microprocessor** architecture consists of two main segments: the **Bus Interface Unit (BIU)** and the **Execution Unit (EU)**. The **BIU** is in charge of interfacing with memory and peripherals, while the **EU** takes care of executing the instructions.

Memory and Segments

- **Extra Segment (ES)**: Allocated 64KB
- **Stack Segment (SS)**: Allocated 64KB

The **address bus** in the 8086 microprocessor is 20 bits wide, allowing it to address **1MB of memory**.

The **data bus** is 16 bits in 8086 and 8 bits in 8088. Additionally, the **control bus** manages signals like **MEMR** (Memory Read), **MEMW** (Memory Write), and I/O signals (**IOR/IOW**).

Dr. Hesham El Zouka

- **Memory Organization**

Let's break it down:

00000

00001

::

00009

0000A

0000B

:::

0000F

00010

00011

:::

:::

000FF ➔ 16*16 = 256

::

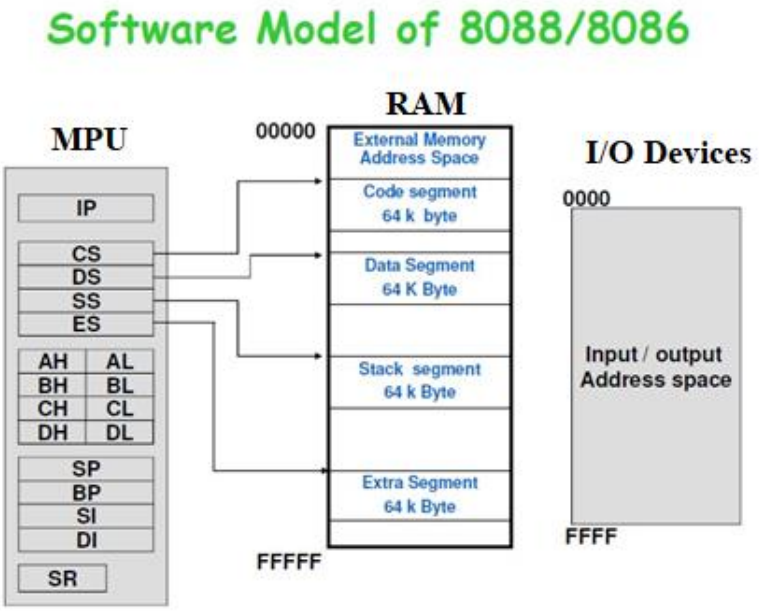
::

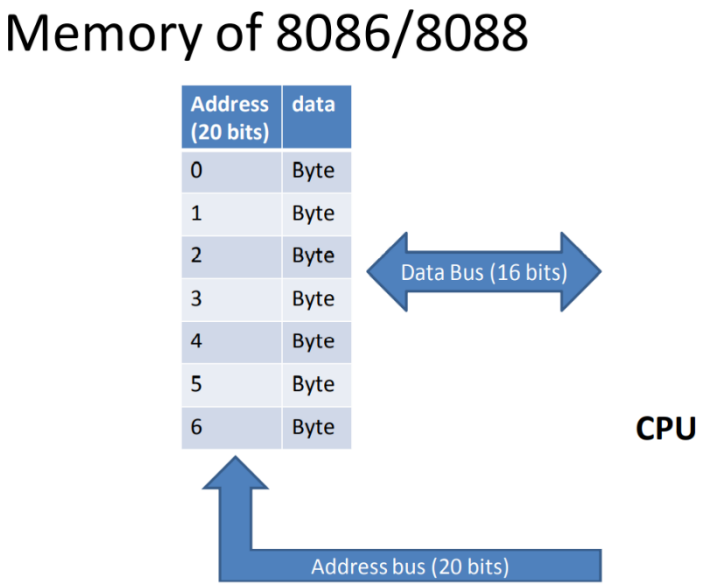
0FFFF ➔ 256*256 = 64KB

:::

::

FFFFF ➔ 64K * 16 = 1MB





- The **address range** spans from **00000** to **FFFFF**, with a maximum memory size of **1MB**.

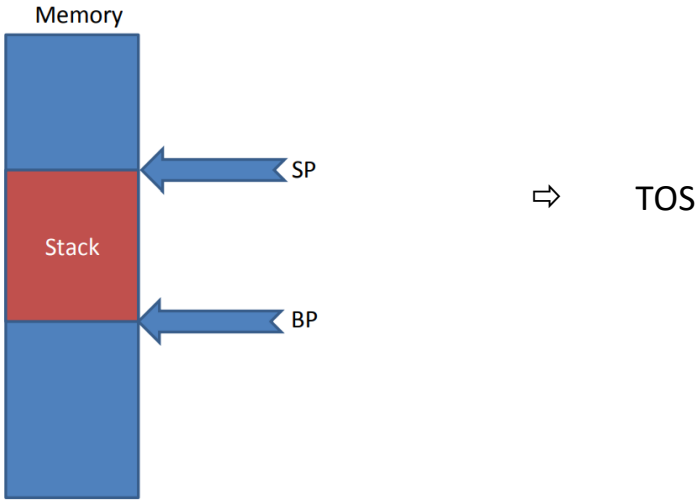
- **Instruction Lifecycle and Pipeline**

  The 8086 uses a **Fetch-Decode-Execute-Store (F|D|E|S)** cycle for processing instructions, implementing pipelining through a **First In, First Out (FIFO)** queue to improve efficiency.

- **Stack Operations**

  The Stack Pointer (SP) and Base Pointer (BP) are 16-bit registers used to manage the stack in memory. The stack operates on a Last In, First Out (LIFO) principle, which is essential for managing function calls, local variables, and saving return addresses.

SP (Stack Pointer) , BP (Base Pointer) => 16 bits



- **Stack Example**

Push AX

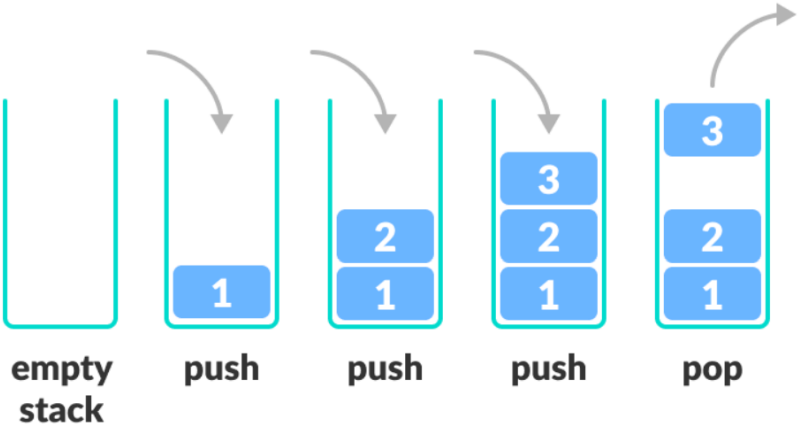
Push BX

Push CX

Pop CX

Pop BX

Pop AX



In this sequence, values are placed onto the stack (push) and then removed (pop) in reverse order.

- ***Applications of the Stack:***

  - *Evaluating expressions with operators and operands.*
  - *Multiplying large numbers, such as:*
    - *Example: 789 * 45 = result with intermediate steps saved on the stack.*

*The stack is also used for **backtracking** in recursive algorithms and problem-solving.*

Address Calculation

Segment registers (CS, DS, ES, SS) help divide memory into different sections. The **physical address** is calculated as: $\text{Physical Address} = \text{Segment Register} \times 10H + \text{Offset (IP)}$

For example, with **CS = 9523** and **IP = FFFF**, the physical address is: $\text{Physical Address} = (9523 \times 10H) + FFFF$

Arrays and Pointers in Assembly

Arrays can be declared and manipulated using index registers in assembly programming.

Array Example:

```
int x[10];
x[0] = 7;
x[1] = 45;
x[2] = -6;
x[3] = 23;
```

Lower address of CS = 95230

Upper address of CS = A522F

Instruction Pointer and Program Counter

The **Instruction Pointer (IP)**, also referred to as the **Program Counter (PC)**, holds the **offset address** of the next instruction to be executed. In the 8086 microprocessor, the IP works alongside the **Code Segment (CS)** register to calculate the physical address of the instruction in memory.

Example: If **CS = 5000H** and **IP = 1234H**, the physical address is calculated as:
$\text{Physical Address} = (CS \times 10H) + IP = (5000H \times 10H) + 1234H = 51234H$

Understanding the role of the Instruction Pointer is critical for writing and debugging programs in assembly language.

Additional Examples:

Segment Register (DS=1771H) and Offset (202AFH)

To calculate both the **Logical Address** and **Physical Address**, as well as to understand the range of the given **Segment Register (DS)**, the following information is provided:

Given:

- **DS (Data Segment) = 1771H**
- **Offset = 202AFH**

Calculations:

1. **Logical Address:** The logical address consists of the segment address and the offset. Therefore, the logical address is written as:

$$\text{Logical Address} = 1771H : 202AFH$$

2. **Physical Address:** The physical address is calculated using the formula:

$$\text{Physical Address} = (\text{Segment} \times 10H) + \text{Offset}$$

Substituting the given values:

$$\text{Physical Address} = (1771H \times 10H) + 202AFH = 17710H + 202AFH = \boldsymbol{379BFH}$$

Therefore, the physical address is:

$$\text{Physical Address} = 1973AFH$$

Range of Segment Register (DS=1771H):

- **Lower Range:** The lowest address is calculated by using the offset **0000H**:

$$\text{Lower Range} = (1771H \times 10H) + 0000H = 17710H$$

- **Upper Range:** The highest address is calculated by using the offset **FFFFH**:

$$\text{Upper Range} = (1771H \times 10H) + FFFFH = 17710H + FFFFH = 2770FH$$

Summary:
Logical Address = ***1771H:202AFH***
Physical Address = ***379BFH***

Range of Segment Register (DS):
Lower Range=***17710H***,
Upper Range=***2770FH***

Question:

In an 8086 microprocessor system, if the Code Segment (CS) register contains the value 2500h and the Instruction Pointer (IP) contains the value 95F3h, calculate the physical address using the formula:

$$\text{Physical Address} = (\text{CS} \times 10h) + \text{IP}$$

Provide the step-by-step process and the final physical address.

Answer:

- CS = 2500h
- IP = 95F3h

Step 1: Multiply CS by 10h (which is 16 in decimal).

$$2500h \times 10h = 25000h$$

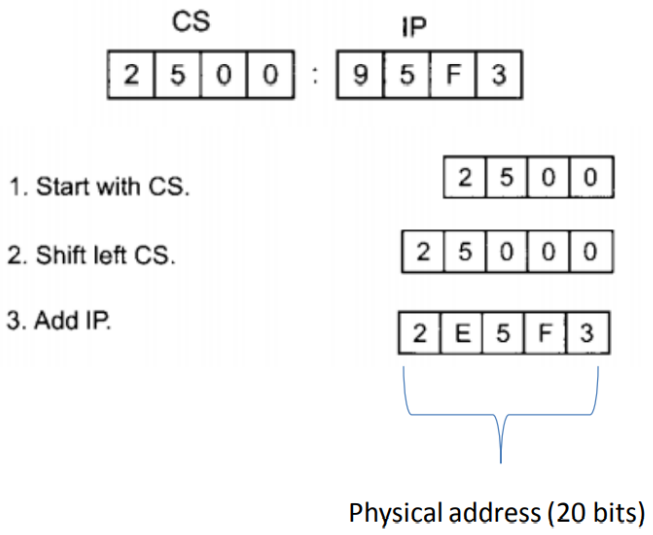
Step 2: Add the value of IP to the result.

$$25000h + 95F3h = 2E5F3h$$

Final Answer:

The **physical address** is **2E5F3h**.

Description:



As an example from the book:

Example 1-1

If CS = 24F6H and IP = 634AH, show:
(a) The logical address
(b) The offset address
and calculate:
(c) The physical address
(d) The lower range
(e) The upper range of the code segment

Solution:
(a) 24F6:634A
(b) 634A
(c) 2B2AA (24F60 + 634A)
(d) 24F60 (24F60 + 0000)
(e) 34F5F (24F60 + FFFF)

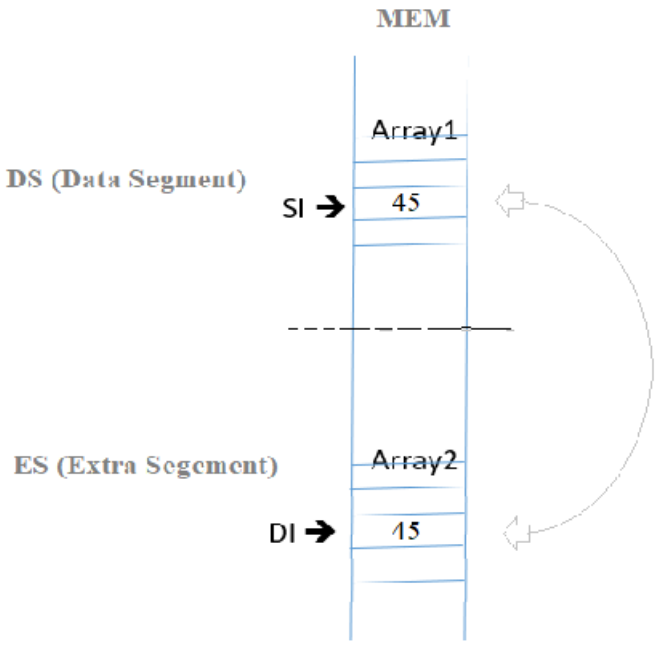
Source Index and Destination Index Pointers

The **Source Index (SI)** and **Destination Index (DI)** pointers play a crucial role in manipulating arrays and string instructions within the 8086/8088 processors. These registers are used to point to locations in the data segment (DS) and extra segment (ES), facilitating the movement of data between arrays or memory blocks.

Example: Working with Arrays

Consider the following array declarations in C-like syntax:

```
int x[10];
x[0] = 7;
x[1] = 45;
x[2] = -6;
x[3] = 23;
// ... other elements
```



Here, we are creating an array `x` of size 10. The values at different indices (positions) in this array are initialized as shown. Similarly, another array `y` of the same size can be declared:

```
int y[10];
```

SI and DI Pointers in Array Operations

In assembly programming for 8086/8088 processors, the **SI** register is typically used to point to the source array, while the **DI** register points to the destination array. For instance, if we need to move elements from array `x` to array `y`, the SI pointer would reference the `x` array (source), and the DI pointer would reference the `y` array (destination).

- **SI (Source Index)**: Points to `Array1` (source array) in the data segment (DS).
- **DI (Destination Index)**: Points to `Array2` (destination array) in the extra segment (ES).

This setup is especially useful in **string instructions**, which can automatically transfer data from the source to the destination.

Direction Flag (DF)

The direction of data movement is controlled by the **Direction Flag (DF)**:

- If **DF = 0**, the processor increments the index registers after each operation, meaning the pointers move forward in memory. This is achieved by the **CLD** (Clear Direction Flag) instruction.
- If **DF = 1**, the processor decrements the index registers, causing the pointers to move backward. This is controlled by the **STD** (Set Direction Flag) instruction.

By manipulating the direction flag, the processor can either process the arrays from the start to the end or in reverse.

Example in Assembly

If we want to move the contents of `x` to `y`, the following operations might occur:

1. **Initialize the Source and Destination Indexes**:

```
mov SI, offset x   ; SI points to the start of array x
mov DI, offset y   ; DI points to the start of array y
```

2. **Clear the Direction Flag to Increment Pointers**:

```
cld            ; Clear DF to ensure SI and DI increment
```

3. **Use String Instructions to Move Data**:

```
movsb   ; Moves a byte from DS:SI to ES:DI, increments both SI and DI
```

Segment Register : Offset Register

Offset Registers for Various Segments

Segment register:	CS	DS	ES	SS
Offset register(s):	IP	SI, DI, BX	SI, DI, BX	SP, BP

The image shows a table mapping **Segment Registers** to their corresponding **Offset Registers** in the 8086/8088 processors:

- **CS (Code Segment)**: Offset register is **IP** (Instruction Pointer).
- **DS (Data Segment)**: Offset registers are **SI**, **DI**, and **BX**.
- **ES (Extra Segment)**: Offset registers are **SI**, **DI**, and **BX**.
- **SS (Stack Segment)**: Offset registers are **SP** (Stack Pointer) and **BP** (Base Pointer).

These segment-offset pairs are used to access different memory segments efficiently in assembly language.

Conclusion:

The handout of this lecture provides a thorough introduction to the 8086/8088 microprocessor architecture, making it an ideal starting point for understanding key concepts. It explains the two primary components—the Bus Interface Unit (BIU) and the Execution Unit (EU)—and their roles in managing memory and executing instructions. The handout also covers important topics such as memory segmentation, how physical addresses are derived, and the instruction cycle. Additionally, it introduces the stack's functionality through the Stack Pointer (SP) and Base Pointer (BP). The document concludes by explaining the role of the Direction Flag (DF), which controls data movement direction, along with the Source Index (SI) and Destination Index (DI) registers, which are used in array and string operations.

Reference Book:

- Mazidi, Muhammad Ali, Janics Mazidi, and Danny Causey. The x86 PC Assembly Language Design, and Interfacing. Pearson Education, 2000.